After Action Report

Hackathon: April 10th, 2025
Breann Larson | Aiden Gallegos

I. Problem

The problem tasked to us was creating a process to check that all sewer lines have contiguous lines. With our elevation data not being up to ideal standards this eliminated us being able to perform checks based on elevations, and without branch versioning we are unable to perform data reviewer checks.

II. Criteria and Problem Mapping

To ensure contiguous lines through all PWSDs sewer mains we decided to create a python script to check for polyline connection to start and end point. This script only allowed one polyline's start point to snap to another polyline's end point. We also added snapping tolerance of 2 feet to snap all unsnapped polylines.

Contiguous Polyline Pseudocode:

1. Identify feature class
2. Get Start and End Points of Polylines within feature class
3. Iterate through polylines in feature class
4. Select Polylines where the start point of one polyline does not meet the end point of the following polyline.
5. Select Polylines where requirement is not met

Vertex Snapping Pseudocode:

1. Select correct polyline layer
2. Verify that the drawing direction of polyline is correct
3. Identify the Start Point & End Point of polyline
4. Confirm that the Start Point of one polyline is snapped within two feet to the End Point of the previous polyline

Steps to Achieve Desired Result:

1. Begin by selecting the appropriate layer and focus on the polyline Start & End Points
2. Use ArcPro Notebook to verify whether information is correct
    a. Driven by pseudocode

III. Tools Used

We approached this problem by trying to use the simplest approach first, which involved researching ArcPro's Data Checker tool. This research enlightened us that Data Checker is only useable within Branch Versioning. The second

approach was to use invert in and invert out elevations or rim elevations of manholes to check for accurate flow direction. However, we felt our elevation data accuracy do not meet the standards to perform this accurately. This brought us to the idea of using the polyline end and start points to check whether or not the lines are contiguous.

We also discussed ArcPros Topology Checks and Validation on Topology. We were able to identify a couple rules needed to guide the code.

1. Point Rules
    a. Must Be Covered By Endpoint Of: Points need to be covered by a Sewer Main
        i. Fix Modify features
    b. Must be Coincident With:
        i. Polyline features
        ii. Snapped to lines
    c. Must be Disjoint: Points in the same feature class cannot overlap
        i. Fix: Modify Feature
    d. Points Must Be Covered By Line: Useful to check that all points are covered by a sewer main
        i. Fix: Validate the unattached points and modify the point
2. Polyline Rules
    a. Must Not Overlap: Ensure that we don't have multiple lines on top of each other
        i. Fix: Remove Overlap
    b. Must Not Have Dangles: Useful for when lines in a feature class or subtype are needed to connect to one another
        i. Fix: Snap if Line needs to be snapped to the next line or trim if it extended past a point.
    c. Endpoint Must Be Covered By: If endpoints of a line are not covered by a point (manholes, sewer valves, etc.)
        i. Fix: Validate the need for a point if not one continuous line needs to be created

Upon further research, validate topology is useable with advanced licensing and the feature layer needs to be published as a web feature layer. Moving forward this is a potential tool we can use once we create a workflow.

Another tool that was tried was Model Builder within ArcPro. With limited knowledge about this tool, we struggled to get a successful result.

IV. Solution

By implementing a python script, the continuity of sewer lines can be checked by Start and End Points. This is helpful when checking if Start & End Points are snapped, and if they are drawn in the correct direction. This solution will ensure accuracy within data and help strengthen the database. This is a step towards creating and maintaining standards within Parker Water data. To address accuracy challenges, a snapping tolerance of two feet was incorporated to align any unsnapped polylines. The code solution involves analyzing the polylines within the feature class to identify discrepancies where Start and End points are not contiguous and correcting these issues. Additionally, the team discussed topology rules in ArcPro, such as ensuring that endpoints of polylines are covered by manholes, and lines must not overlap or have dangles. With the use of topology validation, pseudocode workflows, and python scripting, a solution was able to be achieved.

```
#importing modules

import arcpy
import math



# Input feature class
input_fc = "Feature Class Name”
snap_distance = 2.0    # in the same units as the feature class (e.g., feet)



# Store OIDs that fail the directional check
polylines_to_select = []



# Helper function to calculate the distance between two points
def distance(p1, p2):
    return math.hypot(p1.X - p2.X, p1.Y - p2.Y)



# --- STEP 1: Find bad flow connections ---
print("🔍 Checking for directional flow issues...")
with arcpy.da.SearchCursor(input_fc, ["OID@", "SHAPE@"]) as cursor:
    prev_polyline_end_point = None
    for row in cursor:
        oid = row[0]
        polyline = row[1]
        if polyline is None:
            continue
        start_point = polyline.firstPoint
        end_point = polyline.lastPoint



        if prev_polyline_end_point and (prev_polyline_end_point.X != start_point.X
or prev_polyline_end_point.Y != start_point.Y):
            polylines_to_select.append(oid)



        prev_polyline_end_point = end_point
```

Creates variable for input feature class and for a snapping tolerance distance.

This stores OIDs in a temporary database to pull from later.

This calculates the distance of two points using coordinates and using math.hypot calculates the distance of the line.

Using the SearchCursor function to iterate through the feature classes table and retrieves the OID and Shape as well as the start and end points tied to the OID.

This checks to see if the previous polyline End Point is snapped to the next polyline Start Point.

```
 # --- STEP 2: Select the problematic features ---
if polylines_to_select:
    oid_list = ','.join(map(str, polylines_to_select))
    arcpy.SelectLayerByAttribute_management(
        input_fc, "NEW_SELECTION", f"OBJECTID IN ({oid_list})"
    )
    print(f"Selected {len(polylines_to_select)} polylines with flow mismatch.")
else:
    print("✅ All polylines meet the flow condition.")



# --- STEP 3: Snap endpoints within 2 feet ---
print("\n🔧 Starting vertex snapping...")



# Load all start and end points into memory
endpoints = []
with arcpy.da.SearchCursor(input_fc, ["OID@", "SHAPE@"]) as cursor:
    for row in cursor:
        oid = row[0]
        geom = row[1]
        if geom is None:
            continue
        start = geom.firstPoint
        end = geom.lastPoint
        endpoints.append((oid, start))
        endpoints.append((oid, end))
```

This creates a new layer selection based on the previous criteria.

Pulls start and end points from table in arcpro and stores into temporary database. Using the append method stores start and end point as OID and start/end.

```
  # Snap vertices if they are within 2 feet
with arcpy.da.UpdateCursor(input_fc, ["OID@", "SHAPE@"]) as cursor:
    for row in cursor:
        oid, geom = row
        if geom is None:
            continue
        modified = False

        points = [pt for pt in geom.getPart(0)]
        snapped_points = []

       for point in points:
          snapped = False
          for other_oid, other_point in endpoints:
          if other_oid == oid:
            continue

        if distance(point, other_point) <= snap_distance:
            print(f"Snapping point of OID {oid} to {other_point.X}, {other_point.Y}")
            point.X = other_point.X
            point.Y = other_point.Y
            snapped = True
            break

            snapped_points.append(point)



        if snapped:
            snapped_array = arcpy.Array([arcpy.Point(pt.X, pt.Y) for pt in
snapped_points])
            geom = arcpy.Polyline(snapped_array, geom.spatialReference)
            row[1] = geom
            cursor.updateRow(row)



print("✅ Snapping complete for nearby vertices.")
```

Using the updatecursor method, the code reiterates through the feature class and allows for modification of the polylines.

Get points and checking if points are snapped.

Checks if distance is less then the snapping distance (2 units). Updates new point coordinates if points have been snapped. And updates the point in the temporary list.

Snaps lines to points

IIV. Next Steps

The first step for adjusting this code to better fit our needs is to create a dynamic code that allows user input. This would ask the feature class name, and the user would then input either the path or the feature class name. Making this code dynamic would allow us to create a tool out of our python script.

Aside from further adjusting the code, this hackathon pointed out our need for more accurate data and setting and meeting standards to have higher accuracy within our data.

Overall Take-Aways and Reflection

Aiden

I have learned more about the capabilities that GIS has with automation. The capability to combine data within GIS and python is a useful tool that can ultimately lead to more efficiency within databases. I am trying to use Copilot more; however, I don't find it as useful as ChatGPT. Copilot can be helpful, but it takes more time to ask questions so that Copilot can respond more thoughtfully. This hackathon also showed me that automation is an important aspect to better understanding and analyzing data. GIS is already a great tool for analysis but when code is implemented, there is a next level that is added in accuracy.

Breann

This GIS hackathon reaffirmed the capabilities of using python within a geoprocessing environment. I was able to familiarize myself further with the proper formatting of the syntax and familiarized myself with understanding how a piece of code is performing a process. While implementing code is a great way to do some things within a geoprocessing environment there are often tools that can be used if you know of them. This hackathon showed me the importance of researching different ways of solving a problem and problem mapping before deciding which tool to use. It really showed me the immense number of ways there are to solve different problems. Coding seems to be the most useful when you have a very specific problem you want to solve. Something I always come away from with every GIS Hackathon is a completely new set of skills. It always creates an environment where I can engage my brain in a different way than in our day to day and helps in my growth, whether that's in team communication, coding, problem solving or continued familiarization with GIS.